

P4-AI代码信任度实验报告

团队：海底小纵队

实验日期：2026-05-19

实验功能点：评价提交与信用分计算

AI工具：Claude Code

实验目标：对比AI直出代码与人工修复后代码的质量、稳定性、测试通过率差异

一、实验设计

本实验选取项目核心功能 **评价提交 + 信用分计算** 作为实验对象，通过标准Prompt驱动AI生成完整业务代码，不做任何人工修改；随后按照项目规范、边界场景、异常处理要求进行人工审查与修复；最终通过单元测试验证两种版本代码的通过率，评估AI生成代码的真实信任度。

实验流程：

- 编写精准Prompt，驱动AI生成完整业务代码
- 保留AI原始输出，记录初始状态
- 人工逐项审查代码，标记缺陷与问题
- 修复所有问题，生成合规可用代码
- 运行全套测试用例，对比通过率
- 总结AI代码优缺点，形成信任度结论

二、实验步骤

2.1 原始实验Prompt

为 CampusHub 项目实现评价提交功能，包含信用分计算。

背景

- Spring Boot 项目，使用 MyBatis-Plus
- 相关实体: BizOrder, BizRequirement, BizEvaluation, SysUser
- 已有 Mapper 和各实体
- 已有 BusinessException(code, message) 统一异常类
- 已有 NotificationService 通知服务

需求

1. 用户对已完成订单进行双向评价（发布者评接单者，接单者评发布者）
2. 评分 1-5 星
3. 信用分规则：5星+2，4星+1，3星不变，2星-1，1星-2
4. 信用分范围 0-100
5. 校验：订单已完成、只有参与者可评价、不能重复评价
6. 评价后发送站内通知
7. 评价和信用分更新需要事务保证

请生成完整的 EvaluationServiceImpl 实现代码。

2.2 AI 原始输出代码（未修改版）

```
package com.campushub.evaluation.service.impl;

import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.campushub.common.exception.BusinessException;
import com.campushub.entity.BizEvaluation;
import com.campushub.entity.BizOrder;
import com.campushub.entity.BizRequirement;
import com.campushub.entity.OrderStatus;
import com.campushub.entity.SysUser;
import com.campushub.evaluation.dto.EvaluationSubmitReq;
import com.campushub.evaluation.service.EvaluationService;
import com.campushub.mapper.BizEvaluationMapper;
import com.campushub.mapper.BizOrderMapper;
import com.campushub.mapper.BizRequirementMapper;
import com.campushub.mapper.SysUserMapper;
import com.campushub.notification.NotificationService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class EvaluationServiceImpl implements EvaluationService {

    @Autowired
    private BizOrderMapper bizOrderMapper;
    @Autowired
    private BizRequirementMapper bizRequirementMapper;
    @Autowired
    private BizEvaluationMapper bizEvaluationMapper;
    @Autowired
    private SysUserMapper sysUserMapper;
    @Autowired
    private NotificationService notificationService;

    @Override
    @Transactional
    public void submitEvaluation(Long orderId, Long userId, EvaluationSubmitReq req) {
        // 1. 查询订单
        BizOrder order = bizOrderMapper.selectById(orderId);
        if (order == null) {
            throw new BusinessException(404, "订单不存在");
        }

        // 2. 校验订单状态
```

```
if (!"COMPLETED".equals(order.getStatus())) {
    throw new BusinessException(400, "订单未完成，不能评价");
}

// 3. 查询需求，获取发布者
BizRequirement requirement = bizRequirementMapper.selectById(order.getReqId());
Long publisherId = requirement.getPublisherId();

// 4. 校验评价权限
if (!userId.equals(publisherId) && !userId.equals(order.getReceiverId())) {
    throw new BusinessException(403, "无权评价此订单");
}

// 5. 校验重复评价
LambdaQueryWrapper<BizEvaluation> wrapper = new LambdaQueryWrapper<>();
wrapper.eq(BizEvaluation::getOrderId, orderId)
    .eq(BizEvaluation::getReviewerId, userId);
if (bizEvaluationMapper.selectCount(wrapper) > 0) {
    throw new BusinessException(400, "已评价过此订单");
}

// 6. 保存评价
BizEvaluation evaluation = new BizEvaluation();
evaluation.setOrderId(orderId);
evaluation.setReviewerId(userId);
evaluation.setTargetId(userId.equals(publisherId) ? order.getReceiverId() : publisherId);
evaluation.setStar(req.getStar());
evaluation.setContent(req.getContent());
bizEvaluationMapper.insert(evaluation);

// 7. 计算信用分
Long targetId = evaluation.getTargetId();
SysUser targetUser = sysUserMapper.selectById(targetId);

int scoreChange;
switch (req.getStar()) {
    case 5: scoreChange = 2; break;
    case 4: scoreChange = 1; break;
    case 3: scoreChange = 0; break;
    case 2: scoreChange = -1; break;
    case 1: scoreChange = -2; break;
    default: scoreChange = 0; break;
}

int currentScore = targetUser.getCreditScore();
int newScore = currentScore + scoreChange;
```

```
if (newScore < 0) newScore = 0;
if (newScore > 100) newScore = 100;

targetUser.setCreditScore(newScore);
sysUserMapper.updateById(targetUser);

// 8. 发送通知
notificationService.createNotification(
    targetId,
    "评价通知",
    "您收到了一条" + req.getStar() + "星评价",
    "EVALUATION"
);
}
```

2.3 人工代码审查（问题清单）

问题等级	问题描述	风险影响
严重	未校验 requirement 为空，直接调用 getPublisherId()	触发NPE，系统崩溃
严重	未校验 targetUser 为空，直接操作信用分	触发NPE，系统崩溃
严重	creditScore 为null时自动拆箱，触发NPE	新用户评价必崩
中等	事务注解未指定 rollbackFor ，异常不回滚	数据不一致
中等	错误码使用通用400/403，不符合项目规范	接口异常不统一
中等	通知类型命名不规范，与项目约定不符	通知模块异常
轻微	3星评价无分数变化，仍执行数据库更新	性能冗余

2.4 人工修复方案

1. 增加 requirement 和 targetUser 空值校验
2. 处理 creditScore 为null的默认值（默认100）
3. 完善事务注解：@Transactional(rollbackFor = Exception.class)
4. 统一项目错误码：4003/4004
5. 规范通知类型：EVALUATION_SUBMITTED
6. 分数无变化时跳过数据库更新操作

三、测试对比结果

3.1 核心指标对比

测试指标	AI直出代码	人工修复后
编译通过	✓	✓
正常流程运行	⚠️ 部分可用	✓ 完全可用
测试用例总数	11	11
测试通过数	4	11
测试通过率	36%	100%
NPE风险	✗ 3处	✓ 0处
项目规范匹配	✗ 不匹配	✓ 完全匹配

3.2 测试失败详情（AI直出代码）

- 1. 需求为空导致NPE
- 2. 用户为空导致NPE
- 3. 错误码不匹配
- 4. 通知类型不匹配
- 5. 冗余数据库更新

四、AI代码信任度评估

评估维度	评分(1-5)	评估说明
核心业务逻辑	4	评价流程、信用分计算完全正确
边界/异常处理	2	完全忽略空值场景，存在严重崩溃风险
项目规范适配	2	错误码、通知、事务均不符合项目约定
代码可读性	4	结构清晰，命名规范
生产可用性	1	不可直接上线，必须人工修复

五、实验结论

1. AI 擅长核心业务逻辑实现

对于明确的需求描述，AI可以快速生成正确的主流程代码，大幅提升开发效率。

2. AI 严重缺失防御性编程能力

空值校验、异常处理、边界场景是AI生成代码的最大短板，也是生产环境的核心风险点。

3. AI 无法感知项目内部规范

错误码、枚举值、事务配置、通知格式等项目定制化内容，AI无法自动适配。

4. 最终结论

AI生成代码信任度为36%，不可直接用于生产环境；必须经过人工审查+修复+测试验证后，才能达到100%可用状态。

六、实验总结

AI是高效的代码生成工具，但不是可靠的生产代码提供者。

在软件工程开发中：

- **AI负责：快速实现主流程、模板代码、基础逻辑**
 - **人工负责：边界处理、异常防护、规范适配、测试验证**
- 二者结合是最高效、最安全的开发模式。